

### Step 1: Start Hadoop Services

Open terminal in Ubuntu and run:

**start-dfs.sh**

**start-yarn.sh**

Check whether services are running:

**Jps**

### Step 2: Create Project Folder

**mkdir LogProcessing**

**cd LogProcessing**

### Step 3: Create Input Log File

Create a sample logfile:

**nano logfile.txt**

Add sample data:

ERROR Disk failure

INFO User login

ERROR Memory issue

WARNING CPU high

INFO File opened

ERROR Disk failure

Save file:

- Press CTRL + O
- Press Enter
- Press CTRL + X

Check file contents:

**cat logfile.txt**

### Step 4: Create Java Program

Create Java file:

**nano LogProcessor.java**

Paste this complete code:

```
import java.io.IOException;
```

```
import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;


import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;


import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;


import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;


public class LogProcessor {

    public static class LogMapper
        extends Mapper<Object, Text, Text, IntWritable> {

        private final static IntWritable one = new IntWritable(1);
        private Text word = new Text();

        public void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString();
            String[] words = line.split(" ");

            word.set(words[0]);
            context.write(word, one);
        }
    }
}
```

```
public static class LogReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text key, Iterable<IntWritable> values,
        Context context)
        throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}

public static void main(String[] args)
    throws Exception {

    Configuration conf = new Configuration();

    Job job = Job.getInstance(conf, "log processor");

    job.setJarByClass(LogProcessor.class);

    job.setMapperClass(LogMapper.class);
    job.setReducerClass(LogReducer.class);

    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
}
```

```
FileInputFormat.addInputPath(job, new Path(args[0]));  
FileOutputFormat.setOutputPath(job, new Path(args[1]));  
  
System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
}
```

### **Step 5: Set Hadoop Classpath**

Run:

```
export HADOOP_CLASSPATH=$(hadoop classpath)
```

### **Step 6: Compile Java Program**

```
javac -classpath `hadoop classpath` -d . LogProcessor.java
```

If no error appears, compilation is successful.

### **Step 7: Create JAR File**

```
jar -cvf logprocessor.jar *.class
```

### **Step 8: Create HDFS Input Directory**

```
hdfs dfs -mkdir /input
```

Check:

```
hdfs dfs -ls /
```

### **Step 9: Upload Log File to HDFS**

```
hdfs dfs -put logfile.txt /input
```

Verify:

```
hdfs dfs -ls /input
```

### **Step 10: Remove Old Output Folder (Important)**

Before running MapReduce, delete previous output if it exists:

```
hdfs dfs -rm -r /output
```

If folder does not exist, ignore the error.

### **Step 11: Run MapReduce Program**

**hadoop jar logprocessor.jar LogProcessor /input /output**

Wait until you see:

completed successfully

### **Step 12: View Output Files**

Check output folder:

**hdfs dfs -ls /output**

### **Step 13: Display Final Output**

**hdfs dfs -cat /output/part-r-00000**

Expected output:

ERROR 3

INFO 2

WARNING 1

Explanation:

- ERROR appeared 3 times
- INFO appeared 2 times
- WARNING appeared 1 time

### **Step 14: Copy Output to Local System (Optional)**

**hdfs dfs -get /output**

Now output folder will appear in Ubuntu home directory.